Number Of Binary Tree Topologies

AlgoExpert · Very Hard · Recursion

The problem

- Number of structurally distinct binary trees with n nodes (Catalan).

Intuition

- Pick how many nodes go left; the counts multiply, giving the Catalan numbers.

Brute force vs optimal

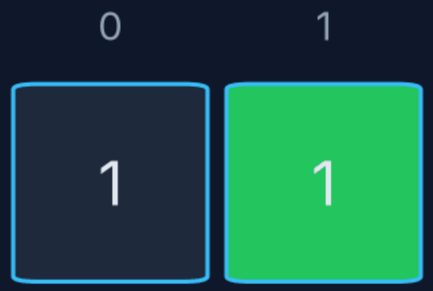
- Recursion without memo → exponential
- DP over sizes → $O(n^2)$
- Closed form: Catalan number

Approach

- 1. ways of zero is one
- 2. For each n, sum left times right counts
- 3. Return ways of n

Worked example

topologies for 0...m nodes

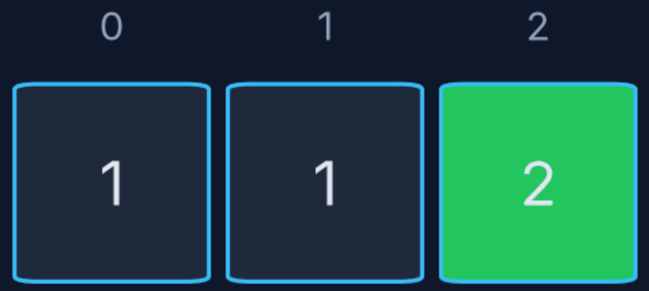


$$c[1] = \Sigma\ c[left] \cdot c[1-1-left] = 1$$

1 nodes → 1

Worked example

topologies for 0...m nodes

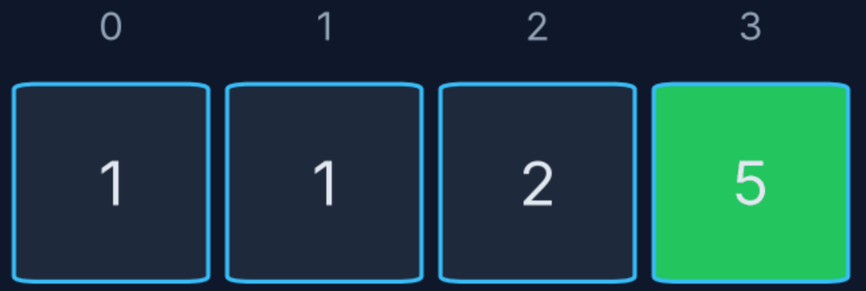


$$c[2] = \Sigma\ c[left] \cdot c[2-1-left] = 2$$

2 nodes → 2

Worked example

topologies for 0...m nodes

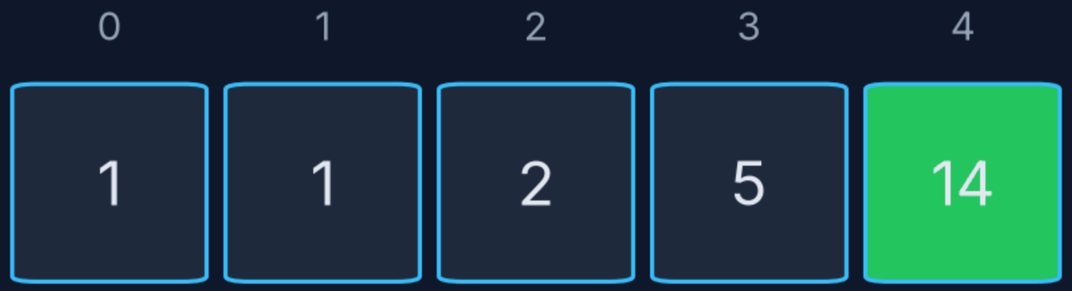


$$c[3] = \Sigma\ c[left] \cdot c[3-1-left] = 5$$

3 nodes → 5

Worked example

topologies for 0...m nodes



$$c[4] = \Sigma\ c[left] \cdot c[4-1-left] = 14$$

4 nodes → 14

Java solution

```java
public class NumberOfBinaryTreeTopologies {
    public static long numberOfBinaryTreeTopologies(int n) {
        long[] c = new long[n + 1];
        c[0] = 1;
        for (int m = 1; m <= n; m++) for (int l = 0; l < m; l++) c[m] += c[l] * c[m - 1 - l];
        return c[n];
    }
}
```

Complexity

- Time: O(n^2)
- Space: O(n)

Edge cases & pitfalls

- C[0] = 1
- Overflow for large n

Interview tips & follow-ups

- Mention Catalan closed form

Recap

- Pick how many nodes go left; the counts multiply, giving the Catalan numbers.
- Complexity: O(n^2)